

TDS200 · Kryssplattform

Session 9

Authentication: Google Sign-In & Security Rules

Google sign-in · Protected Routes · Security Rules

Cross-Platform Mobile Development

What you'll learn today

- Google sign-in via OAuth 2.0 (expo-auth-session)
- Where the Web client ID, consent screen and redirects fit
- Protect routes with Expo Router groups + redirects
- Cloud Security Rules



Guarded app

Unauthenticated users are redirected to the login screen.



Project step

Add Google sign-in to your app.
Lock the whole app behind auth and harden the backend rules.

The 12 sessions

1 Introduction & Foundations

2 Components & Navigation (Expo Router)

3 Components, State & Local Storage

4 Forms, Modals & Creating Data

5 Firebase Firestore CRUD

6 Firestore CRUD & Camera

7 Firebase Storage, Camera & Image Picker

8 Firebase Authentication & Context

9 Google sign-in, Firebase Rules & Guards

10 NativeWind & Geolocation

11 Google Maps & Queries

12 Exam & Wrap-Up

What is new in the codebase?

- We have modified `auth.tsx` and `api/authApi.ts` to integrate Google Sign-in.
- We have added `redirect.tsx` under the root folder to help Web Google Sign-in.
- We have setup configurations in Google Cloud Console to enable Google Sign-in.
- We have inserted Google API keys in `.env` for Google Sign-in.
- We have moved both `(tabs)` and `postDetails` folder into `(protected)` folder.
- We have created `(protected)/_layout.tsx` to protect all routes inside `(protected)` and redirect unauthenticated user to the login page.
- We have modified the root layout to simply contain the stack navigation.
- We have configured camera and image picker access in `app.json`.

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Google OAuth 2.0 Provider, consent, redirect URIs
0:45-1:00	Break —
1:00-1:40	Route guards + security rules Protected groups, redirects, rules
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

Google sign-in via OAuth 2.0



Firebase console

Enable Google as a sign-in provider



Google Cloud console

Same project; OAuth consent screen



Web client ID

Auto-generated; used in your app code



Your Expo app

expo-auth-session runs the login flow

Integrate Google sign-in in Firebase

- Step 1: Create a Firebase project (which you already have)
- Step 2: Enable Google sign-in in Firebase
- Step 3: Configure OAuth consent screen (one-time setup)
- Step 4: Configure Web OAuth client and add authorized redirect Uris.
- Step 5: Configure iOS OAuth client and add authorized redirect Uris.
- Step 6: Update .env file with Web OAuth client and iOS OAuth client Ids. Also update app.json
- Step 7: Integrate Google sign-in in your Expo app code.

Create a Firebase Project

Step 1: create a Firebase project

- When you create a Firebase project (Firebase console -> add project -> create), a corresponding **Google Cloud project** is also created under the same project ID.
- Firebase is built on top of Google Cloud infrastructure.
- This means the same project can be viewed and managed from both:
 - The Firebase Console, which focuses on Firebases such as Authentication, Firestore, Storage, and hosting.
 - The Google Cloud Console, which exposes the underlying Google Cloud project, APIs, IAM permissions etc

Firebase console->my-awesome-project->project settings

Project name my-awesome-project 

Project ID  my-awesome-project-704b5

Google console->all projects. Choose the same as Firebase project

Select a project  [New project](#)

Search projects and folders

[Recent](#) [Starred](#) [All](#)

	Name	Type	ID	
✓	 my-awesome-project 	Project	my-awesome-project-704b5	

Google Cloud Console Setup

Steps 2–5: enable Google in Firebase, configure OAuth consent screen, then configure the Web and iOS OAuth clients

1



Enable Google in Firebase

Firebase Console → Authentication → Sign-in method → Google → Enable → Save
Firebase auto-creates a “Web client (auto created by Google Service)” in the same Cloud project.

1.5



Configure the consent screen (don't skip)

Google Cloud Console → APIs & Services → OAuth Consent Screen → Google Auth Platform → Branding + Audience
Audience = **External**, add app name + support email, keep the `openid` · `email` · `profile` scopes, and add yourself as a **test user** — otherwise sign-in returns *access_blocked / 403*.

2



Configure the Web OAuth client

Cloud Console → APIs & Services → Credentials → edit the auto-created Web client
JS origins: `http://localhost:8081+http://127.0.0.1:8081` · **Redirect:** `http://127.0.0.1:8081/redirect` → copy **WEB_CLIENT_ID**

3



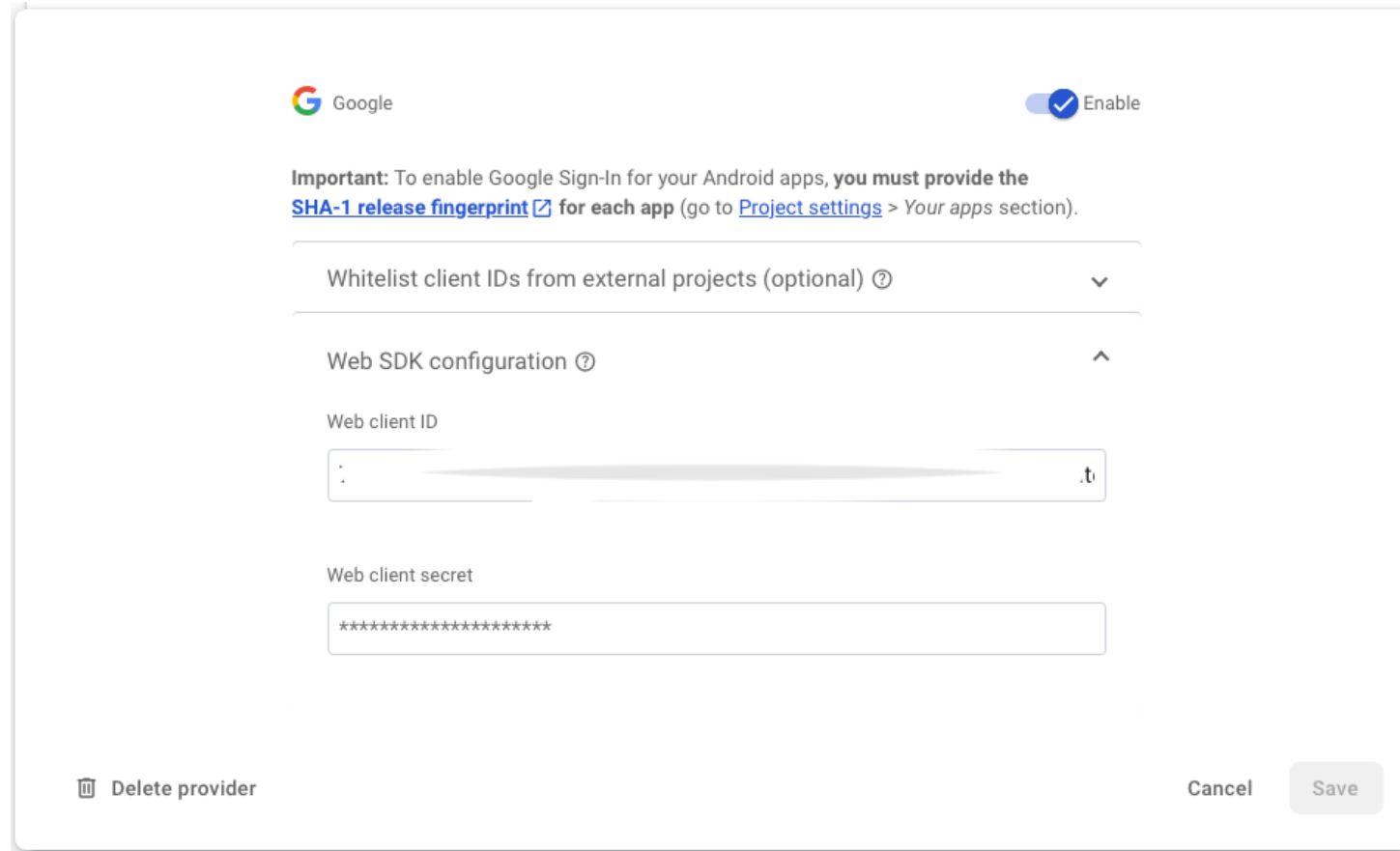
Create an iOS OAuth client

Credentials → + Create Credentials → OAuth client ID → Application type: iOS
Declare a bundle ID (required, no wildcards). Without App Check it isn't enforced at runtime — the **reversed-client-ID URL scheme** is the redirect, so it needn't match `app.json` exactly. → copy **IOS_CLIENT_ID**

Google Cloud Console Setup

Steps 2: enable Google in Firebase

- Go to Firebase Console -> Authentication -> Sign-in method -> Google, enable the Google provider.
- Firebase uses an underlying OAuth 2.0 Web Client for Google Sign-in.
- In Firebase Authentication, this appears as the Web Client ID, as shown in the picture.



The screenshot shows the 'Google' provider configuration page in the Firebase Authentication console. At the top, the Google logo is on the left and a toggle switch labeled 'Enable' is on the right, which is currently turned on. Below this, an important note states: 'Important: To enable Google Sign-In for your Android apps, you must provide the [SHA-1 release fingerprint](#) for each app (go to [Project settings](#) > Your apps section)'. The main configuration area includes a dropdown menu for 'Whitelist client IDs from external projects (optional)' with a downward arrow. Below this is a section for 'Web SDK configuration' with an upward arrow. Inside this section, there is a label 'Web client ID' followed by a text input field containing a partially visible client ID ending in '.it'. Below that is a label 'Web client secret' followed by a password input field filled with asterisks. At the bottom of the form, there are three buttons: 'Delete provider' with a trash icon, 'Cancel', and 'Save'.

Google Cloud Console Setup

Steps 2: enable Google in Firebase

- The same OAuth client is stored in the corresponding Google Cloud project under: Google Cloud Console -> API & Services -> Credentials -> OAuth 2.0 Client IDs.
- Therefore, the Firebase Web Client ID and the corresponding Google Cloud OAuth 2.0 Web Client ID refer to the same OAuth client and should have the same value.
- In the normal Firebase setup, you do not need to manually create the Web OAuth Client ID. Firebase auto-generates it when Google Sign-in is enabled. Manual creation is normally only needed for repairing a missing or invalid OAuth client.

Name *

Web client (auto created by Google Service)

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.



The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Additional information

Client ID	7617c4f581742-nga1c34q120bntjnc0tq11s3p0ve7n.apps.googleusercontent.com
Creation date	October 6, 2025, 12:53:44 PM GMT+2
Last used date	June 23, 2026 (Note: this data could be delayed by a day or more.)

Google Cloud Console Setup

Step 3: configure OAuth consent screen

- Go to Google Cloud console -> APIs & Services -> OAuth Consent Screen, you will be led to **Google Auth Platform**. Choose the same project ID as Firebase project.
- Under Branding, provide App name, user support email, and developer contact information.
- Under Audience, choose who can use the app:
 - Internal: only users in your Google Workspace organization
 - External: users with Google accounts outside your organization

Google Auth Platform / Overview / Create branding

Overview | Project configuration

1 App Information

App name *
my-awesome-app
The name of the app asking for consent

User support email *
For users to contact you with questions about their consent. [Learn more](#)

Next

2 Audience

3 Contact Information

4 Finish

Create Cancel

Google Auth Platform / Overview / Create branding

Overview | Project configuration

Branding

Audience

Clients

Data Access

Verification Center

✓ App Information

✓ Audience

✓ Contact Information

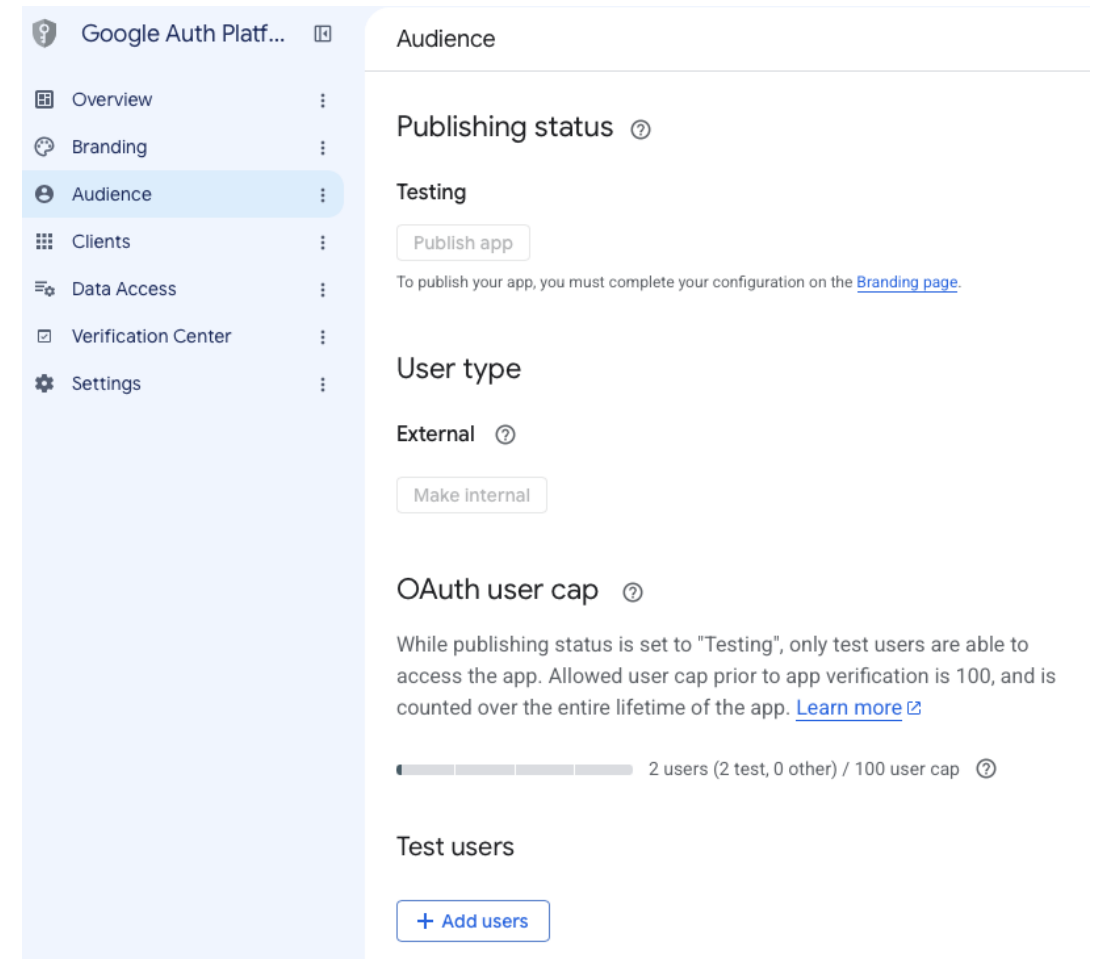
✓ Finish

Create Cancel

Google Cloud Console Setup

Step 3: configure OAuth consent screen

- If your Google OAuth app is external and in Testing mode, you should add a test user.
 - It is recommended to use Testing mode in your exam. Add your own Google account under Test users.
 - **IMPORTANT FOR THE EXAM:** the examiner's Google account must be added as a Test user if you want the examiner can verify you google Sign-in function.
- tds.exam.test@gmail.com
- If direct testing by the examiner is not possible, you should at minimum demonstrate the complete Google Sign-in process clearly in your submitted video, if you have implemented Google Sign-in as an extended function.
 - You **MUST NOT** share your Google OAuth client secret or service-account private key with examiner!



Google Cloud Console Setup

- Go to Google Auth Platform -> Clients. The auto-generated Client ID provides details of Client ID, and Client secret. You will use the Client ID in your code for Google sign-in.

Google Auth Platform / Clients / Client: 70c...

Overview

Branding

Audience

Clients

Data Access

Verification Center

← Client ID for Web application Delete

Name *

Web client (auto created by Google Service)

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Additional information

Client ID	70c600071c...
	nga1c34q2...
Creation date	October 6, 2025 at 12:53:44 PM GMT+2
Last used date	October 6, 2025 (Note: this data could be delayed by a day or more.)

Google Cloud Console Setup

Step 4: configure the Web OAuth clients

- Go to Google Cloud console -> Google Auth Platform -> Clients -> OAuth 2.0 Client Ids
- Click Web client (auto created by Google service).
- Under Authorized redirect URIs, add every redirect URI that your app may use during Google OAuth flow, e.g.
 - <http://127.0.0.1:8081/redirect>
- Note at these URIs must match exactly what you use in redirectUri inside your Expo app.
- Under Authorized JavaScript origins, add below URIs, this is which browser origins are allowed to start OAuth requests:
 - <http://127.0.0.1:8081/>
 - <http://localhost:8081/>

Name *
Web client (auto created by Google Service)

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#) if.

Authorized JavaScript origins ⓘ

For use with requests from a browser

URI 1 *
<http://localhost>

URI 2 *
<http://localhost:8081>

URI 3 *
<http://127.0.0.1:8081>

URI 4 *
<http://127.0.0.1:8081>

[+ Add URI](#)

Authorized redirect URIs ⓘ

For use with requests from a web server

URI 1 *
<https://127.0.0.1:8081/auth/handler>

URI 2 *
<https://127.0.0.1:8081/auth/handler>

URI 3 *
<http://127.0.0.1:8081/redirect>

[+ Add URI](#)

Google Cloud Console Setup

Step 4: configure the Web OAuth clients

- Firebase automatically creates PROJECT_ID.firebaseio.com as the default authentication domain. Firebase Authentication uses this domain internally during OAuth sign-in.
- For Google authentication, Firebase registers a callback URL such as https://my-awesome-project-xxxx.firebaseio.com/_auth/handler with Google OAuth. We normally do not need to use this callback URL directly in your Expo app code.
- A self-defined redirect URL, such as <http://127.0.0.1/redirect> is used when your application manages part of the OAuth flow yourself.



Name *

Web client (auto created by Google Service)

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.



The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#) [2].

Authorized JavaScript origins ⓘ

For use with requests from a browser

URIs 1 *

<http://localhost>

URIs 2 *

<http://localhost:8081>

URIs 3 *

<https://my-awesome-project-xxxx.firebaseio.com>

URIs 4 *

<http://127.0.0.1:8081>[+ Add URI](#)

Authorized redirect URIs ⓘ

For use with requests from a web server

URIs 1 *

https://my-awesome-project-xxxx.firebaseio.com/_auth/handler

URIs 2 *

<https://my-awesome-project-xxxx.firebaseio.com/lecture10-google-signin-comment>

URIs 3 *

<http://127.0.0.1:8081/redirect>[+ Add URI](#)

Google Cloud Console Setup

Step 5: configure the iOS OAuth clients

- Go to Google Cloud console -> Google Auth Platform -> Clients -> Create Client
- Select to create iOS client
- Enter the bundle identifier from your expo app app.config.json
- After creating the iOS OAuth client, Google generates an iOS Client ID, i.e., 123-abc.app.googleusercontent.com, and an iOS URL scheme, with com.googleusercontent.apps.123-abc

←

Client ID for iOS

Delete

Name *

iOS client 1

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Bundle ID *

com.tds200.session9

The bundle identifier as listed in the app's Info.plist file

App Store ID

The app's App Store ID, if the app is published in the Apple App Store

Team ID

The unique 10-character string generated by Apple that's assigned to your team

Use this client for an AI-powered agent

Additional information

Client ID

703706990742-rtdp77bku23u6onin9rmvk5bidf5hvjvleusercontent.com

iOS URL scheme

com.googleusercontent.apps.703706990742-rtdp77bku23u6onin9rmvk5bidf5hvjv

Creation date

June 7, 2026, 4:53:51 PM GMT+2

Last used date

October 2, 2026 (Note: this data could be delayed by a day or more.)

Inactive OAuth clients are subject to deletion if they are not used for 6 months. You will be notified of deletion due to inactivity, and can restore clients up to 30 days after deletion. [Learn more](#)

Google Identity for iOS

Google Cloud Console Setup

Step 5: configure the iOS OAuth clients

- Expo Web uses the Web OAuth client.
- Native iOS build on simulators or physical phone requires configuring the iOS OAuth client and matching bundle ID.
- To test Google Sign-in on an iOS simulator, we need a native iOS build of the Expo app.
npx expo run:ios
- This command creates and prepares the native iOS/ folder, builds the iOS application, and installs it on the iOS simulator.
- Check app.json. When you create expo app, Expo generates a default bundle identifier. But you should explicitly set it in app.json to match your Google Cloud iOS OAuth client.

```
newAppEnabled": true,  
"ios": {  
  "bundleIdentifier": "com.tds200.session9",
```

- The bundle identifier uniquely identifies that application. We will use exactly the same bundle ID when configuring Google OAuth.
- Why **npx expo run:ios**? When you run with npx expo start, your app is running inside the Expo Go app, so Google does not see it as your own real iOS app. That causes Google login fail..

Redirect URIs

iOS derives its URI from the reversed client ID; Web must use an exact 127.0.0.1 match

 **IOS** · REVERSED CLIENT ID + ":/"

CLIENT ID

123-abc.apps.googleusercontent.com

REVERSE THE DOTS

com.googleusercontent.apps.123-abc

APPEND :/

com.googleusercontent.apps.123-abc:/



auth.tsx

```
makeRedirectUri({
  native: iosClientId.split(".")
    .reverse().join(".") + "://"
});
```



app.config.json

```
"ios": { "infoPlist": {
  "CFBundleURLTypes": [{ "CFBundleURLSchemes":
    ["com.googleusercontent.apps.123-abc"] }]
} }
```

npm expo run:ios is required — custom URL schemes only work in native builds, not Expo Go.



WEB · EXACT LOCAL MATCH

redirect URI

http://127.0.0.1:8081/redirect



Use 127.0.0.1, not localhost — Google Cloud Console requires an exact string match.

Wiring It Up — .env and app.json

Steps 6: enter the client IDs into .env, register the reversed iOS ID (iOS URL scheme) as a URL scheme manually in app.json

Update .env file

```
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=<web id>
EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID=<ios id>
```

Manually register CFBundleURLSchemes in app.json

```
"ios": {
  "infoPlist": {
    "CFBundleURLTypes": [{
      "CFBundleURLSchemes": [
        "com.googleusercontent.apps.123-abc"
      ]
    }]
  }
}
```



- **You now have both client IDs:** the **Web ID** is used when the app runs in the browser/Expo web, and the **iOS ID** is used when the app runs as a native iOS build.
- `npx expo start` starts the Metro development server and serves as JavaScript/TypeScript bundle to Expo Go or to an already-built development app.
- You need `npx expo run:ios` when the app depends on native configuration that must be compiled into the iOS app, such as: **a custom OAuth redirect URL scheme which is required by Google Sign-in**; changes to the bundle identifier; native config plugins, etc.

Wiring It Up — app.json

Steps 6: other configurations in app.json

- Run `npx expo run:ios` to enable Google Sign-in in iOS simulator.
- What does this command do? It creates ios and android folders if they do not pre-exist, builds them and installs the app.
- Remember to put ios and android folders in `.gitignore`!
- If you already have ios and android folders, and you change native settings in `app.json`, i.e., schemes, plugins, the Google client etc, `npx expo run:ios` will not regenerate them.
- In this case, run following two commands:
 - `npx expo prebuild --clean`, then
 - `npx expo run:ios`.
- Expo go app is not used here since you did not run the command `npx expo start`!

Wiring It Up — app.json

Steps 6: other configurations in app.json

- For camera (not supported in simulator) and image picker, native permissions must now be configured in app.json!
- Recall from Session 7, camera permission in Expo requires two layers:
 - Native permission declaration in app.json, and
 - Runtime permission when the feature is used.
- With Expo Go, this native configuration is already included. However, we are now using native build (due to Google Sign-in), we must configure the permissions ourselves.

THEORY · PERMISSIONS

Two layers of Camera Permission in Expo

- Camera permissions in Expo requires two layers:
 - **Native permissions declaration (build time).**
 - Required for standalone/native builds. Configured through app.json and Expo config plugins.
 - The operating system must know in advance that your app intends to use the camera.
 - iOS: requires a usage string in Info.plist (e.g. `NSCameraUsageDescription`).
 - Android: requires camera permissions in AndroidManifest.xml. Expo automatically generates the appropriate native configuration when building the app.
 - With Expo Go, this native configuration is already included, so you do not need to add camera plugin.
 - **Runtime permission (when the feature is used).**
 - Still required in both Expo Go and your own native build.
 - Request permission in JavaScript, e.g., with `useCameraPermissions()` / `requestPermission()`.
 - Ask when the user actually opens the camera feature, not necessarily at app launch.
 - Handle granted / denied / restricted states gracefully.

Two layers of permission for Native

```
"ios": {
  "supportsTablet": true,
  "infoPlist": {
    "NSCameraUsageDescription": "We need access to your camera.",
    "NSPhotoLibraryUsageDescription": "We need access to your photos to select images.",
    "NSPhotoLibraryAddUsageDescription": "We save photos you take to your library.",
    "NSMicrophoneUsageDescription": "We use the microphone when recording video."
  }
},
"android": {
  "adaptiveIcon": {
    "backgroundColor": "#E6F4FE",
    "foregroundImage": "./assets/images/android-icon-foreground.png",
    "backgroundImage": "./assets/images/android-icon-background.png",
    "monochromeImage": "./assets/images/android-icon-monochrome.png"
  },
  "edgeToEdgeEnabled": true,
  "predictiveBackGestureEnabled": false,
  "permissions": [
    "CAMERA",
    "READ_MEDIA_IMAGES",
    "READ_MEDIA_VIDEO",
    "READ_EXTERNAL_STORAGE"
  ]
},
```

UX

Example of app.json

Two layers of permission - Native

```
"plugins": [  
  "expo-router",  
  [  
    "expo-image-picker",  
    {  
      "photosPermission": "The app accesses your photos to let you share them with your friends."  
    }  
  ],  
  [  
    "expo-camera",  
    {  
      "cameraPermission": "We need to access your camera so that you can take beautiful pictures!"  
    }  
  ],  
  [  
    ]  
  ]  
]
```

UX

Example of app.json

Sign in with Google

terminal

```
npx expo install expo-auth-session \
expo-web-browser
```

Web

- Add authorized redirect URIs in Google Cloud.
- Use the Web client ID in the request.
- Google redirects to /redirect with an id_token.
- Exchange the Google credential into Firebase.

iOS

- Register an iOS app in Firebase / Google Cloud with a bundle ID.
- Copy the generated iOS client ID into IOS_CLIENT_ID.
- Use the iOS client ID in the Google auth request.
- Expo AuthSession redirects back to the app using the reversed client ID URL scheme.
- Expo exchanges the authorization code automatically.
- Read response.authentication.idToken.Exchange the Google idToken into Firebase.

Sign in with Google

Steps 7: Integrate Google Sign-in in your expo app code

Code handles Web and iOS differently:

- Web: id_token flow.
- For iOS (equivalent flow for Android and iOS), Expo Auth session uses code (PKCE).

```
// iOS redirect URI: reverse the client ID dots and append :/ to form a custom URL scheme.  
// e.g. "12345-abc.apps.googleusercontent.com" → "com.googleusercontent.apps.12345-abc:/"  
const iosRedirectUri = iosClientId  
  ? makeRedirectUri({ native: iosClientId.split(".").reverse().join(".") + "://" })  
  : undefined;  
  
// Web: id_token implicit flow – popup redirects to /redirect, which posts GOOGLE_AUTH_SUCCESS.  
// iOS: code flow (PKCE) – hook auto-exchanges the code; idToken in response.authentication.  
const [request, response, promptAsync] = Google.useAuthRequest({  
  clientId: webClientId,  
  iosClientId: iosClientId,  
  redirectUri: Platform.OS === "web" ? "http://127.0.0.1:8081/redirect" : iosRedirectUri,  
  responseType: Platform.OS === "web" ? "id_token" : "code",  
  scopes: ["openid", "profile", "email"],  
  extraParams: { prompt: "select_account" },  
});
```

Platform Differences – Two Different OAuth Flows

iOS and Web must use different response types



iOS

PKCE code flow

```
responseType: "code"
```

- ✓ expo-auth-session auto-exchanges the code internally
- ✓ idToken lives at `response.authentication?.idToken`
- ✓ We then convert the returned Google ID token to Firebase credential and pass it to Firebase.



Web

implicit flow

```
responseType: "id_token"
```

- ✓ expo-auth-session opens a popup
- ✓ Polls roughly every 200 ms to detect when the popup reaches the redirect URI
- ✓ Captures the `id_token` from the URL hash (popup mechanics on the pitfalls slide)

Sign in with Google

Steps 7: Integrate Google Sign-in in your expo app code

For iOS (equivalent flow for Android and iOS), Expo Auth session handles most of the OAuth work.

- The expo-auth-session uses PKCE by default.
- Google first returns an authorization code to the app's custom redirect URI.
- The expo-auth-session automatically exchanges that code for tokens (PKCE) by default.
- The resulting authentication response then contains the google ID token in: `response.authentication.idToken`
- Firebase then signs in using `signInWithGoogleIdToken()`.

```
// iOS: hook auto-exchanges the code; idToken is in response.authentication.
useEffect(() => {
  if (response?.type !== "success") return;
  if (Platform.OS === "web") return;

  const idToken = response.authentication?.idToken;
  if (idToken) {
    signInWithGoogleIdToken(idToken).catch(() => showToast("Google sign-in failed."));
  } else {
    showToast("Google sign-in failed: no id_token");
  }
}, [response]);
```

Sign in with Google

Steps 7: Integrate Google Sign-in in your expo app code

For Web, id_token flow is used.

- Google opens the sign-in popup. After authentication, Google redirects back to the configured web redirect URI, such as <http://127.0.0.1:8081/redirect>.
- The redirect page completes the popup flow and sends the authentication result back to the main application.
- The returned Google idToken is then used to create the Firebase credential.
- The credential is passed to Firebase. Firebase verifies it and signed the user in.

```
// Web: listener must be attached on mount, before the popup sends the message.
useEffect(() => {
  if (Platform.OS !== "web") return;

  const handlePopupMessage = (event: MessageEvent) => {
    if (event.data?.type === "GOOGLE_AUTH_SUCCESS" && event.data.idToken) {
      signInWithGoogleIdToken(event.data.idToken).catch(() =>
        showToast("Google sign-in failed.")
      );
    }
  };

  window.addEventListener("message", handlePopupMessage);
  return () => window.removeEventListener("message", handlePopupMessage);
}, []);
```

SESSION 9 • GOOGLE OAUTH

Sign in with Google

Steps 7: Integrate Google Sign-in in your expo app code

- This page is a special redirect page used only for Google Sign-In on the web.
- User clicks "Sign in with Google" -> Google login popup opens -> Google redirects to /redirect -> RedirectHandler runs -> redirect.tsx extracts id_token from the URL -> sends idToken back to the main window using window.opener.postMessage() -> Closes popup.
- The main window receives GOOGLE_AUTH_SUCCESS -> signInWithGoogleIdToken -> Firebase signInWithCredential -> User is signed in

```
// app/redirect.tsx
import * as WebBrowser from "expo-web-browser";
WebBrowser.maybeCompleteAuthSession(); // must run first

import { useEffect } from "react";

// Popup-only relay page: just hands the id_token to the opener window, which
// owns the actual signInWithGoogleIdToken call (see auth.tsx's message listener).
// Do not call signInWithGoogleIdToken here too – that would sign in twice.
export default function RedirectHandler() {
  useEffect(() => {
    const hashParams = new URLSearchParams(window.location.hash.substring(1));
    const idToken = hashParams.get("id_token");

    if (idToken && window.opener) {
      window.opener.postMessage({ type: "GOOGLE_AUTH_SUCCESS", idToken }, "*");
      window.close();
    } else {
      window.location.replace("/");
    }
  }, []);

  return <div>Completing login...</div>;
}
```

A complete flow

Google sign-in work flow for Web

User taps Sign in with Google
↓
promptAsync()
↓
Google login page opens
↓
User selects Google account
↓
Google authenticates the user
↓
Google redirects to
http://127.0.0.1:8081/redirect
↓
Google returns an ID token directly
↓
ID token is available in the auth response
↓
signInWithGoogleIdToken(idToken)
↓
GoogleAuthProvider.credential(idToken)
creates a Firebase credential
↓
signInWithCredential(auth, credential)
↓
Firebase signs the user in
↓
Firebase auth state updates
↓
App shows authenticated screens

Google sign-in work flow for iOS

User taps Sign in with Google
↓
promptAsync()
↓
Google login page opens
↓
User selects Google account
↓
Google authenticates the user
↓
Google redirects back to the iOS app
↓
Google returns an authorization code
↓
expo-auth-session automatically exchanges
the code using PKCE
↓
ID token becomes available at
response.authentication?.idToken
↓
GoogleAuthProvider.credential(idToken)
creates a Firebase credential
↓
signInWithCredential(auth, credential)
↓
Firebase signs the user in
↓
Firebase auth state updates
↓
App shows the authenticated screens

ROUTER GUARD

Protected route group

Recall Session 8, the imperative guard



app/_layout.tsx

```
// Session 8 - imperative
function RootNavigator() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const segments = useSegments();

  useEffect(() => {
    if (loading) return;
    const onAuthScreen = segments[0] === "auth";
    if (!user && !onAuthScreen) router.replace("/auth");
    else if (user && onAuthScreen) router.replace("/");
  }, [user, loading, segments]);
  // ...
}
```



Runs after render

The effect fires whenever user, loading, or segments change — then redirects. This can cause a brief flash of the wrong screen.



Fragile

This is a single check, but now the root layout has to know which screens are public and which are private. Every time you add a public screen, you have to update the logic. If you get it wrong, you can redirect loops.

Route Groups — Folders in () Parentheses

A group organises files without adding anything to the URL



HOW THE FOLDER MAPS TO THE URL

app/ **(protected)** /postDetails/[id].tsx



/postDetails/[id]

app/ **(protected)** / **(tabs)** /index.tsx



/



() adds no URL segment

Any folder wrapped in parentheses is a route group. It groups files together for organisation and shared layout — but the parentheses never appear in the URL.
(protected) is not a magic word. It can be (private) or other words.



_layout.tsx is the group's gate

Every screen in the group renders through the group's _layout.tsx. That single entry point is the perfect place to put one auth guard for all of them.

Protected route group

Session 9, the declarative guard

```
(protected) /_layout.tsx

// app/(protected)/_layout.tsx
export default function ProtectedLayout() {
  const { user, loading } = useAuth();

  if (loading) {
    return <ActivityIndicator />;    // wait for session
  }
  if (!user) {
    return <Redirect href="/auth" />; // not signed in
  }

  return (
    <Stack>
      <Stack.Screen name="(tabs)"
        options={{ headerShown: false }} />
      <Stack.Screen name="postDetails/[id]"
        options={{ headerBackTitle: "Back" }} />
    </Stack>
  );
}
```



(protected) router guard

With (protected), the folder decides

app/

└─ auth.tsx ← outside = public

└─ (protected)/

└─ _layout.tsx ← one guard

└─ ... ← inside = private

One guard in (protected)/_layout.tsx protects every screen inside the folder. A new screen in the folder is protected automatically.

This means the file location determines the access rule, and there is no need to write separate if statement for navigation.

auth.tsx sits outside the protected group and the guard.

The name (protected) does not appear in the URL or route path. This also makes it easier to understand which screens are public or private just by looking at the file tree.

Protected route group

The RootLayout is now clean. No auth logic at the top — the (protected) group owns it

```
// Root layout: wraps the whole app in AuthProvider and declares the top-level screens.  
// Auth guard logic lives in app/(protected)/_layout.tsx — this file has no auth logic.  
export default function RootLayout() {  
  return (  
    <AuthProvider>  
    <Stack>  
      <Stack.Screen name="(protected)" options={{ headerShown: false }} />  
      <Stack.Screen name="auth" options={{ headerShown: false }} />  
      <Stack.Screen name="redirect" options={{ headerShown: false }} />  
    </Stack>  
  </AuthProvider>  
);  
}
```

Protected route group

The RootLayout can still support granular protection

- A (protected) route group can still support granular protection.
- In option 2, the group name does not force every route inside it to require authentication.
- The group layout checks both
 - the user's authentication state, and
 - the current route
- Some routes can remain public.
- The main point of (protected) is organization and scoping.

```
// Option 1: every screen in this group requires sign-in (original behavior).
//const isGuestAllowed = false;

// Option 2: guests can also view the Home tab; everything else still requires sign-in.
const segments = useSegments();
const onHomeScreen = segments[1] === "(tabs)" && segments[2] === undefined;
const isGuestAllowed = onHomeScreen;

if (loading) {
  return (
    <View style={styles.loading}>
      <ActivityIndicator size="large" color="#007AFF" />
    </View>
  );
}

if (!user && !isGuestAllowed) {
  return <Redirect href="/auth" />;
}

return (
  <Stack>
    <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
    <Stack.Screen name="postDetails/[id]" options={{ headerBackTitle: "Back" }} />
  </Stack>
);
}
```

Protected route group

The RootLayout can still support granular protection

- We can see we have two stacks and one is inside another.
- This works and is a common Expo Router pattern.
- You can get the same result with a single stack. Stack.Protected is added in SDK 53, and we can use it.
- The guard moves into the root layout and (protected)/_layout.tsx goes away.
- We will leave this feature to interested audience to explore.



```
app/_layout.tsx      ← Stack #1 (root): (protected), auth, redirect
└─ (protected)/_layout.tsx ← Stack #2: (tabs), postDetails/[id] + auth guard
    └─ (tabs)/_layout.tsx  ← Tabs: index, profilePage
```



```
// app/_layout.tsx
function RootStack() {
  const { user, loading } = useAuth();
  if (loading) return <Spinner />;
  return (
    <Stack>
      <Stack.Protected guard={!user}>
        <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
        <Stack.Screen name="postDetails/[id]" options={{ headerBackTitle: "Back" }} />
      </Stack.Protected>
      <Stack.Protected guard={!user}>
        <Stack.Screen name="auth" options={{ headerShown: false }} />
      </Stack.Protected>
      <Stack.Screen name="redirect" options={{ headerShown: false }} />
    </Stack>
  );
}
```

Cloud Security Rules

- Rules run on Firebase, not the client — the real gate.
- Require authentication to both read/write.
- We will enforce more granular security rules in both Cloud and client in later sessions.

Firestore rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /posts/{postId} {
      allow read, write: if request.auth != null;
    }

    match /comments/{commentId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Storage rules

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /{allPaths=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

ASSIGNMENT 9

Router guards

YOUR TASKS

In this session, we add Firestore security rules that the server enforces auth and ownership on every write. Please extend current security rules by:

- Review the Firestore API functions and find functions that do not check user sign-in. Add a client-side authentication guard to those functions. If there is no signed-in user, throw an error.
- Update deletePost so that a user can only delete their own post. Hint: remove Edit and Delete buttons if the post is not owned by the logged in user. Apply client-side check.
- Update the comment delete feature so that a user can only delete its own comments. Hint: Remove the comment delete button if the comment is not created by the logged in user. Apply client-side check.

DELIVERABLE

A secured app where only authenticated users can perform Firestore write operations, users can delete only their own posts, and comment delete buttons are visible only to the comment author.

EST. EFFORT

≈ 1–3 hours